



BUG BOUNTY FIELD GUIDE

Shodan Dorking

Advanced Search Filters, Real-World Dorks &
Automation Techniques for Bug Bounty Hunters

- 50+ Search Filters
- 2025 Edition

Table of Contents

1. Introduction to Shodan

2. Core Search Filters

2.1 Network & Location Filters

2.2 Service & Product Filters

2.3 HTTP / Web Filters

2.4 SSL Certificate Filters

3. Bug Bounty Dorks by Category

3.1 Exposed Admin Panels

3.2 PII & Data Leaks

3.3 Vulnerable Services & CVEs

3.4 Cloud & DevOps Exposure

3.5 IoT & Industrial Systems

4. Targeting Specific Organizations

5. Shodan CLI & API Automation

6. Integration with Bug Bounty Workflow

7. Best Practices & Legal

1. Introduction to Shodan

Shodan is the world's first search engine for internet-connected devices. Unlike Google, which indexes web pages, Shodan indexes service banners, open ports, and metadata from billions of devices worldwide — from web servers and routers to industrial control systems and webcams.



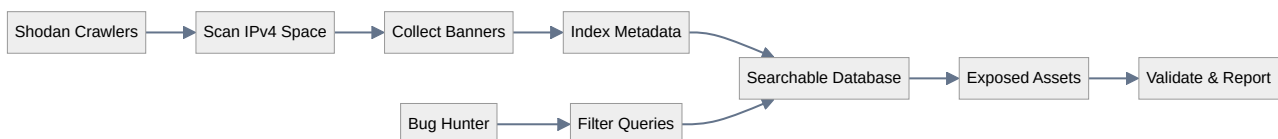
For bug bounty hunters, Shodan is a goldmine. It enables **passive reconnaissance** — discovering exposed assets without ever sending a packet to the target. This is critical for programs that restrict active scanning or when you need to map an organization's full attack surface quietly. 🕵️

Why Shodan Matters for Bug Bounty

Passive Discovery: Find exposed assets without active scanning. **Version Fingerprinting:** Identify outdated software with known CVEs. **Shadow IT:** Discover forgotten cloud instances and dev servers. **Scope Validation:** Verify which IPs and hostnames belong to your target organization.

How Shodan Works

Shodan continuously scans the entire IPv4 address space, sending connection requests to common ports and recording the responses (banners). These banners reveal software names, versions, configuration details, and sometimes even internal paths. All this data is indexed and searchable through filters.



Free vs Membership Features

Table 1 Shodan Account Tiers

Feature	Free	Membership (\$49/mo)
Search results	10 pages	Unlimited
API queries	100 credits/month	Unlimited
SSL analysis	Basic	Full certificate chains
Image data	No	Screenshots of HTTP services

Data export	Limited	JSON/CSV/XML bulk download
Monitor alerts	No	Up to 65 IPs

2. Core Search Filters

Shodan's power comes from its extensive filter library. Understanding these filters is the key to crafting precise queries that surface high-value targets. 🔍

2.1 Network & Location Filters

LOCATION & NETWORK

Table 2 Geographic & Network Filters

Filter	Description	Example
country	2-letter country code	country:"US"
city	City name	city:"London"
geo	Lat/long coordinates	geo:37.7749, -122.4194
net	IP range or CIDR	net:192.168.0.0/16
ip	Specific IP address	ip:8.8.8.8
asn	Autonomous System Number	asn:AS15169
isp	Internet Service Provider	isp:"Amazon.com"
org	Organization name	org:"Microsoft"

2.2 Service & Product Filters

SERVICE FINGERPRINTING

Table 3 Service & Product Filters

Filter	Description	Example
--------	-------------	---------

product	Software product name	product:"Apache"
version	Software version	version:"2.4.41"
port	Open port number	port:3389
os	Operating system	os:"Windows Server 2019"
devicetype	Type of device	devicetype:"webcam"
vuln	Known CVE ID	vuln:"CVE-2021-44228"
cpe	Common Platform Enumeration	cpe: cpe:/a:apache:httpd
banner	Raw banner text search	banner:"220 FTP"

2.3 HTTP / Web Filters

WEB APPLICATION

Table 4 HTTP & Web Filters

Filter	Description	Example
http.title	HTML page title	http.title:"Login"
http.status	HTTP status code	http.status:302
http.html	Content of HTML page	http.html:"password"
http.server	Server header value	http.server:"nginx"
http.component	Web technology	http.component:"jQuery"
http.favicon.hash	Hash of favicon.ico	http.favicon.hash:-335242539
http.headers	Raw HTTP headers	http.headers:"X-Frame-Options"
http.redirect	Redirect destination	http.redirect:"https://auth.target.com"

2.4 SSL Certificate Filters

CERTIFICATE OSINT

Table 5 SSL/TLS Certificate Filters

Filter	Description	Example
<code>ssl.cert.subject.cn</code>	Common Name in cert	<code>ssl.cert.subject.cn:"*.target.com"</code>
<code>ssl.cert.issuer.cn</code>	Certificate issuer	<code>ssl.cert.issuer.cn:"DigiCert"</code>
<code>ssl.cert.expired</code>	Expired certificates	<code>ssl.cert.expired:true</code>
<code>ssl.version</code>	SSL/TLS version	<code>ssl.version:"ssl2"</code>
<code>ssl.chain_count</code>	Chain length	<code>ssl.chain_count:1</code>
<code>ssl.cert.serial</code>	Serial number	<code>ssl.cert.serial:1234567890</code>
<code>ssl</code>	Has SSL enabled	<code>ssl:true</code>

Boolean Operators

Use `+` to include terms, `-` to exclude, and `"` for exact phrases. Example: `apache -nginx port:443 country:"DE"` finds Apache servers on port 443 in Germany that are NOT nginx.

3. Bug Bounty Dorks by Category

This section contains battle-tested Shodan dorks organized by the type of vulnerability or exposure they surface. Each dork includes the exact query and what to look for in results. 

3.1 Exposed Admin Panels

HIGH-IMPACT DORKS

Administrative interfaces are juicy targets. When exposed to the internet without proper access controls, they can lead to full system compromise.

ADMIN **Apache Tomcat Manager**

Exposed Tomcat manager interfaces often allow WAR file deployment for RCE.

```
http.title:"Apache Tomcat" http.html:"manager"
```

ADMIN **Jenkins Dashboard**

Unprotected Jenkins instances expose build pipelines, secrets, and often allow script console access.

```
http.title:"Dashboard [Jenkins]" http.component:"jenkins"
```

ADMIN **phpMyAdmin Login**

phpMyAdmin provides full database control. Default or weak credentials = full data compromise.

```
http.title:"phpMyAdmin" http.html:"pma_username"
```

ADMIN **Cisco ASDM / VPN**

Cisco Adaptive Security Device Manager and VPN endpoints with known CVEs.

```
http.title:"Cisco ASDM" ssl.cert.subject.cn:"*.target.com"
```

ADMIN **Kubernetes Dashboard**

Exposed Kubernetes dashboards may allow pod execution and cluster takeover.

```
http.title:"Kubernetes Dashboard" http.html:"Kubernetes"
```

ADMIN **Docker Registry**

Unauthenticated Docker registries leak container images and source code.

```
http.title:"Docker Registry" port:5000 http.status:200
```

ADMIN Grafana Login

Grafana dashboards may expose server metrics, logs, and admin panels.

```
http.title:"Grafana" http.component:"grafana"
```

ADMIN Kibana Elasticsearch

Exposed Kibana interfaces reveal log data and may allow query execution on Elasticsearch.

```
http.title:"Kibana" port:5601 http.status:200
```

3.2 PII & Data Leaks

SENSITIVE DATA

Shodan can locate publicly exposed directories, files, and databases containing personally identifiable information. These findings often lead to P1/P2 severity reports. ●

PII Open Directory Listings

Directory listings may expose backups, config files, databases, and source code.

```
hostname:"target.com" http.title:"Index of /"
```

PII Exposed .env Files

Environment files contain API keys, database credentials, and secrets.

```
http.html:"DB_PASSWORD" http.html:"APP_KEY" http.title:"200"
```

PII Exposed MongoDB

Unauthenticated MongoDB instances expose entire databases over the internet.

```
product:"MongoDB" port:27017 -authentication
```


PII ElasticSearch Open

Elasticsearch without auth allows reading, modifying, or deleting all indexed data.

```
port:9200 product:"Elastic" http.title:"200 OK"
```

PII Git Config Exposure

Exposed .git/config and .git/HEAD files leak repository URLs and sometimes credentials.

```
http.html:"[core]" http.html:"repositoryformatversion"
```

PII SQL Backup Files

Database dumps and backup files found in open directories or misconfigured servers.

```
http.html:".sql" http.title:"Index of" http.html:"dump"
```

3.3 Vulnerable Services & CVEs

CVE HUNTING

Shodan's `vuIn:` filter (membership-only) and product/version fingerprinting make it possible to find instances of known vulnerabilities at scale. 🌟

CVE Log4j (CVE-2021-44228)

Find services running Log4j versions vulnerable to the infamous RCE bug.

```
vuIn:"CVE-2021-44228" org:"target"
```

CVE Apache Struts (CVE-2017-5638)

Apache Struts with known OGNL expression injection vulnerability.

```
product:"Apache Struts" vuIn:"CVE-2017-5638"
```

CVE PAN-OS XSS (CVE-2025-0133)

Palo Alto Networks GlobalProtect with reflected XSS in getConfig.esp endpoint.

```
os:"PAN-OS" ssl.cert.subject.CN:"target.com"
```

CVE F5 BIG-IP (CVE-2022-1388)

F5 BIG-IP iControl REST auth bypass — critical RCE without authentication.

```
http.title:"BIG-IP" product:"BIG-IP" port:443
```

CVE GitLab CE RCE (CVE-2021-22205)

GitLab Community Edition with ExifTool RCE via image upload.

```
http.title:"GitLab" http.html:"GitLab Community Edition"
```

CVE Spring4Shell (CVE-2022-22965)

Spring Framework with Java 9+ running on Tomcat — WAR file write RCE.

```
http.html:"Spring Boot" http.component:"Spring"
```

3.4 Cloud & DevOps Exposure

CLOUD INFRA

Cloud misconfigurations are among the highest-impact bug bounty findings. Shodan surfaces S3 buckets, exposed APIs, dev environments, and container orchestration systems. ☁

CLOUD AWS S3 Buckets

Misconfigured S3 buckets may allow public listing, upload, or full data access.

```
http.title:"Bucket" http.html:"ListBucketResult" http.html:"s3.amazonaws.com"
```

CLOUD Exposed Redis

Redis without authentication exposes cached data and may allow config rewrite.

```
product:"Redis" port:6379 -auth
```

CLOUD Swagger / API Docs

Exposed Swagger UI and API documentation reveal endpoints and sometimes test tokens.

```
http.title:"Swagger UI" http.html:"swagger-ui"
```

CLOUD GraphQL Playground

GraphQL introspection enabled allows full schema discovery and potential data over-extraction.

```
http.title:"GraphQL Playground" http.html:"graphql"
```

CLOUD AWS EC2 User-Data

Metadata service exposure (IMDSv1) can leak IAM credentials and user-data scripts.

```
http.html:"iam/security-credentials" http.title:"404"
```

CLOUD MinIO Storage

MinIO object storage interfaces may expose buckets without authentication.

```
http.title:"MinIO Browser" http.html:"MinIO"
```

3.5 IoT & Industrial Systems

IOT / ICS

Internet of Things and Industrial Control Systems (ICS) often have default credentials, unpatched firmware, and no encryption. These are critical findings with real-world safety implications. 🚨

IOT **Unprotected Webcams**

IP cameras with no authentication — privacy breach and physical security risk.

```
title:"webcam" port:80,443,8080 -auth
```

IOT **CCTV DVR Systems**

Digital video recorders with known default passwords and remote access.

```
http.title:"DVR" http.html:"login" product:"DVR"
```

IOT **HP iLO / iDRAC**

Server management interfaces with known CVEs and default credentials.

```
http.title:"iLO" ssl.cert.subject.cn:"ilo" port:443
```

IOT **Modbus / SCADA**

Industrial control protocols exposed to the internet — critical infrastructure risk.

```
port:502 product:"Modbus"
```

IOT **VNC without Authentication**

VNC servers with authentication disabled allow full remote desktop access.

```
"authentication disabled" port:5900,5901
```

IOT **Siemens / Schneider**

Industrial devices from major vendors with known vulnerabilities and default configs.

```
product:"Siemens" OR product:"Schneider" port:102,502,44818
```

4. Targeting Specific Organizations

The `org:` and `hostname:` filters are the bread and butter of targeted Shodan reconnaissance for bug bounty. Here's how to map an organization's full attack surface. 🗺️

Organization Enumeration Strategy

STEP-BY-STEP TARGETING

```
# Step 1: Identify the exact org name
org:"Example Company"

# Step 2: Discover associated hostnames
org:"Example Company" ssl.cert.subject.cn:"*.example.com"

# Step 3: Find all HTTP titles for that org
org:"Example Company" http.title:"Login"

# Step 4: Look for redirects to internal services
org:"Example Company" http.redirect:"*.internal.example.com"

# Step 5: Check for recent exposures (last 7 days)
org:"Example Company" after:"2025-05-01"

# Step 6: Find services on non-standard ports
org:"Example Company" -port:80,443,8080,8443
```

Hostname-Based Recon

RECON Wildcard Subdomain Hunt

Find all indexed subdomains for a target domain using SSL certificate common names.

```
ssl.cert.subject.cn:"*.target.com"
```

RECON Specific Hostname Services

Query specific hostname patterns to find dev, staging, or API endpoints.

```
hostname:"dev.target.com" http.status:200,301,302
```

RECON IP to Hostname Mapping

Resolve IPs to hostnames and find certificate-validated subdomains.

```
ip:"203.0.113.0/24" ssl.cert.subject.cn:"target.com"
```

Favicon Hash Hunting

Favicon Hash Technique

Many web applications use unique favicon.ico files. Computing the MurmurHash3 of a favicon lets you find **all instances** of that application across the internet, including forgotten dev servers and shadow IT deployments.

FAVICON HASH WORKFLOW

```
# 1. Get favicon from target
curl -s https://target.com/favicon.ico | python3 -c "
import sys, mmh3, base64
img = sys.stdin.buffer.read()
print(mmh3.hash(base64.encodebytes(img)))
"

# 2. Search Shodan by hash
http.favicon.hash:YOUR_HASH_VALUE

# 3. Combine with org filter for precision
org:"Target Company" http.favicon.hash:-335242539
```

5. Shodan CLI & API Automation

Manual web searches are fine for spot checks, but bug bounty hunters need automation. Shodan's CLI and Python API enable scalable, repeatable reconnaissance. 🤖

Shodan CLI Setup

CLI INSTALLATION

```
# Install via pip
pip install shodan

# Initialize with your API key
shodan init YOUR_API_KEY

# Verify connection
shodan info
```

Essential CLI Commands

Table 6 Shodan CLI Commands

Command	Purpose	Example
<code>shodan search</code>	Execute search queries	<code>shodan search apache</code>
<code>shodan host</code>	View host details	<code>shodan host 8.8.8.8</code>
<code>shodan count</code>	Count results only	<code>shodan count apache</code>
<code>shodan download</code>	Bulk data export	<code>shodan download apaches apache</code>
<code>shodan parse</code>	Parse downloaded JSON	<code>shodan parse apaches.json.gz</code>
<code>shodan stats</code>	Aggregated statistics	<code>shodan stats apache</code>
<code>shodan alert</code>	Network monitoring	<code>shodan alert create "MyNet" 1.2.3.0/24</code>

Python API Scripting

PYTHON API — TARGETED ENUMERATION

```
import shodan

# Initialize client
api = shodan.Shodan("YOUR_API_KEY")

# Search for target org with pagination
try:
    results = api.search("org:\"Target Company\" http.title:\"Login\"")
    print(f"Results: {results['total']}")
    for result in results['matches']:
        ip = result['ip_str']
        port = result['port']
        title = result.get('http', {}).get('title', 'N/A')
        hostnames = result.get('hostnames', [])
        print(f"{ip}:{port} | {title} | {hostnames}")
except shodan.APIError as e:
    print(f"Error: {e}")
```

PYTHON API — CVE MONITOR

```
import shodan
from datetime import datetime, timedelta

api = shodan.Shodan("YOUR_API_KEY")

# Find newly indexed vulnerable targets
after_date = (datetime.now() - timedelta(days=7)).strftime("%Y-%m-%d")

query = f'org:"Target" vuln:"CVE-2021-44228" after:"{after_date}"'

try:
    for result in api.search_cursor(query):
        print(f"[{result['ip_str']}] {result['hostnames']}")
except shodan.APIError as e:
    print(e)
```

One-Liner Automation

CLI ONE-LINERS


```
# Export IPs for nmap scanning
shodan search org:"Target" --fields ip_str | tee target_ips.txt

# Get all hostnames for a target
shodan search ssl.cert.subject.cn:"*.target.com" --fields hostnames | sort -u

# Bulk JSON export for analysis
shodan download target_data "org:\"Target Company\""
shodan parse target_data.json.gz --fields ip_str,port,hostnames,title

# Monitor your own netblock
shodan alert create "BugBounty-Target" 192.0.2.0/24
shodan alert triggers # list available triggers
shodan alert enable ALERT_ID new_cve # alert on new CVEs

# Search with pagination (100 results)
shodan search "product:jenkins org:\"Target\"" --limit 100 --fields ip_str,port,title
```

6. Integration with Bug Bounty Workflow

Shodan is most powerful when integrated into a broader reconnaissance pipeline. Here's how professional hunters chain Shodan with other tools. [🔗](#)

The Shodan + Subfinder Pipeline

CHAINED RECON PIPELINE

```
# Phase 1: Passive subdomain discovery
subfinder -d target.com -all -silent | tee subs.txt

# Phase 2: Resolve subdomains to IPs
cat subs.txt | dnsx -a -silent | tee subs_with_ips.txt

# Phase 3: Extract unique IPs
cut -d' ' -f1 subs_with_ips.txt | sort -u | tee unique_ips.txt

# Phase 4: Query Shodan for each IP
for ip in $(cat unique_ips.txt); do
    shodan host "$ip" >> shodan_host_data.txt
done

# Phase 5: Filter for interesting services
grep -E "(Jenkins|Tomcat|phpMyAdmin|Kibana|Grafana|Elastic)" shodan_host_data.txt

# Phase 6: Validate with httpx
cat subs.txt | httpx -title -status-code -tech-detect -o live_services.txt
```

Shodan + Nuclei for CVE Validation

CVE VALIDATION PIPELINE

```
# Step 1: Find targets with a specific product
shodan search "product:Apache http.title:\"Tomcat\"" \
  --fields ip_str | tee tomcat_ips.txt

# Step 2: Probe for live HTTP services
cat tomcat_ips.txt | httpprobe -p http:8080 | tee live_tomcat.txt

# Step 3: Run Nuclei templates for Tomcat CVEs
cat live_tomcat.txt | nuclei -t cves/ -tags tomcat -severity critical,high
```

Continuous Monitoring Script

DAILY MONITOR CRON SCRIPT

```
#!/bin/bash
# monitor.sh – Daily Shodan recon monitor

TARGET="org:\"Target Company\""
API_KEY="YOUR_KEY"
OLD="shodan_last.json"
NEW="shodan_current.json"

# Pull current state
shodan search "$TARGET" --limit 1000 -O "$NEW"

# Compare with previous run
diff "$OLD" "$NEW" | grep "^>" | tee shodan_new_findings.txt

# Send notification if new assets found
if [ -s shodan_new_findings.txt ]; then
  notify -bulk -data shodan_new_findings.txt -id shodan-alerts
fi

# Rotate files
mv "$NEW" "$OLD"
```

Scope & Rate Limiting

Bug bounty programs often restrict automated scanning. **Shodan is passive** — it does not send packets to the target directly, making it ideal for scope-restricted programs. However, always verify program rules. Some prohibit using Shodan data for targets outside scope or for commercial purposes.

7. Best Practices & Legal

Validation Checklist

Table 7 Before You Report

Step	Action	Why It Matters
1	Confirm asset belongs to target	False positives waste everyone's time
2	Verify exposure is in scope	Out-of-scope reports may be rejected
3	Check if already reported	Duplicates don't earn bounties
4	Validate exploitability	Confirmed findings get higher payouts
5	Document Shodan query used	Shows your methodology to triagers
6	Take screenshots with timestamps	Proves the finding is current

Writing a Strong Report

Report Template for Shodan Findings

Title: [P1] Exposed Jenkins Dashboard at IP 203.0.113.42

Description: During passive reconnaissance, I identified a publicly accessible Jenkins instance belonging to [Target].

Shodan Query Used: `org:"Target Company" http.title:"Dashboard [Jenkins]"`

Evidence: [Screenshot from Shodan + direct browser access]

Impact: Unauthenticated access to Jenkins allows viewing build logs (potential secret exposure), running builds, and potentially accessing the Script Console for RCE.

Remediation: Restrict Jenkins to internal network or enforce strong authentication + MFA.

Legal & Ethical Guidelines

Important Reminders

Do not exploit without authorization. Shodan reveals exposures, but accessing systems without permission may be illegal under CFAA, Computer Misuse Act, and similar laws. **Only report to programs where you have explicit scope. Never modify data, upload files, or execute commands on discovered systems. Respect responsible disclosure timelines.**

Recommended Resources

- **Shodan** — shodan.io (Official search engine & documentation)
- **Shodan Python Library** — github.com/achillean/shodan-python
- **Shodan Dorks Cheat Sheet** — github.com/jakejarvis/shodan-dorks
- **ExploitDB** — exploit-db.com (Map CVEs to exploits)
- **Nuclei Templates** — github.com/projectdiscovery/nuclei-templates
- **Intigriti Blog** — intigriti.com/blog (Shodan + Censys guides)